

Informe de Seguridad: Análisis de Vulnerabilidades en un Servidor Django Local

Fecha: [30 / 08 / 2024]

Nombre del Responsable: Luis, Andrés Y Sebastián

Nombre del Proyecto: ThesisTrack

Introducción

Se realizó un análisis de seguridad utilizando la herramienta Nikto para identificar vulnerabilidades en un servidor Django local que está en desarrollo emulando un entorno universitario y con datos sensibles de los usuarios. El servidor se encuentra corriendo en `http://127.0.0.1:8000`, y el análisis fue ejecutado desde la terminal del sistema utilizando el comando Nikto.

Proceso Realizado

El análisis de vulnerabilidades se llevó a cabo mediante el siguiente comando en la terminal:

```
bash
Copiar código
godprogrammer@godprogrammer-VivoBook-ASUSLaptop-X421DA-M413DA ~> nikto -h
http://127.0.0.1:8000 -output resultados2.txt
```

Nikto es una herramienta de análisis de vulnerabilidades en servidores web que realiza una revisión completa de configuraciones inseguras, problemas de seguridad comunes y posibles vulnerabilidades.

Resultados del Análisis

El análisis reveló las siguientes vulnerabilidades y problemas de seguridad:

- 1. Listado de Directorios:** Nikto detectó que el servidor permite el listado de directorios. Esto significa que un atacante podría acceder y visualizar la estructura de archivos y directorios en el servidor, lo que podría exponer archivos sensibles.
- 2. Vulnerabilidades de XSS (Cross-Site Scripting):** Nikto identificó posibles vulnerabilidades de XSS en URLs que aceptan parámetros sin la debida sanitización, permitiendo la inyección de scripts maliciosos que podrían comprometer la seguridad de los usuarios.

Descripción de Vulnerabilidades y Posibles Soluciones

A continuación, se describen las vulnerabilidades detectadas y se plantean posibles soluciones para mitigarlas:

1. Prevención del Listado de Directorios

- **Descripción:** El listado de directorios permite a los usuarios visualizar el contenido de directorios en el servidor web. Esto puede exponer archivos confidenciales, scripts, o configuraciones que un atacante podría aprovechar.
- **Posible Solución:**

- **Apache:** Se debe configurar el archivo de configuración de Apache para que la directiva `Options -Indexes` esté activada, evitando que los directorios sean indexados y listados.
- **Nginx:** En el archivo de configuración de Nginx, es necesario establecer `autoindex off;` para deshabilitar el listado de directorios.

2. Prevención de XSS (Cross-Site Scripting)

- **Descripción:** Las vulnerabilidades de XSS permiten a un atacante inyectar scripts maliciosos en una aplicación web. Estos scripts pueden ejecutarse en los navegadores de otros usuarios, robando información sensible o realizando acciones en su nombre.
- **Posible Solución:**
 - **Sanitización de Entradas:** Se deben sanitizar todas las entradas de usuario antes de ser procesadas o mostradas en la página. Django, por defecto, escapa las variables en las plantillas para evitar inyecciones XSS.
 - **Implementación de Cabeceras de Seguridad:** Se recomienda configurar las siguientes cabeceras de seguridad en Django para mitigar XSS:

```
python
Copiar código
# settings.py
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_BROWSER_XSS_FILTER = True
X_FRAME_OPTIONS = 'DENY'
```

- **Content-Security-Policy (CSP):** Implementar CSP puede restringir los orígenes desde donde se permiten cargar scripts, lo que proporciona una defensa adicional contra XSS:

```
python
Copiar código
# Ejemplo de CSP usando django-csp
CSP_DEFAULT_SRC = (''self'',)
CSP_SCRIPT_SRC = (''self'', 'https://trustedscripts.example.com')
```

3. Configuraciones de Seguridad en Django

- **Descripción:** La configuración predeterminada de Django puede no ser segura para un entorno de producción. Aspectos como la habilitación de `DEBUG`, la configuración de `ALLOWED_HOSTS`, y la seguridad de las cookies y conexiones, son críticos.
- **Posible Solución:**
 - **Deshabilitar DEBUG en Producción:** Se debe deshabilitar el modo de depuración en producción cambiando `DEBUG = False`.
 - **Configuración de ALLOWED_HOSTS:** Se recomienda configurar `ALLOWED_HOSTS` con los dominios que se espera que accedan al servidor, limitando así el acceso no autorizado.

- **Habilitar HTTPS y Cookies Seguras:** Se deben activar configuraciones que requieran el uso de HTTPS y asegurar que las cookies sean transmitidas de forma segura:

```
python
Copiar código
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

4. Sanitización de Parámetros de URL

- **Descripción:** Parámetros de URL que no son debidamente sanitizados pueden ser utilizados para realizar ataques XSS, SQL Injection, y otros tipos de inyecciones.
- **Posible Solución:**
 - **Validación y Sanitización:** Todos los parámetros recibidos en la URL deben ser validados y sanitizados antes de ser procesados o utilizados en consultas.

5. Deshabilitación de Funcionalidades Innecesarias

- **Descripción:** Mantener activadas características y aplicaciones que no se usan puede aumentar la superficie de ataque.
- **Posible Solución:**
 - **Revisión y Deshabilitación:** Se recomienda revisar las aplicaciones y configuraciones activas en Django, deshabilitando aquellas que no sean necesarias en el entorno de producción.

Conclusión

El análisis de seguridad con Nikto reveló varias vulnerabilidades que podrían comprometer la seguridad del servidor Django local. Aunque no se han implementado soluciones, este informe plantea medidas prácticas para mitigar los riesgos identificados. Es esencial considerar estas recomendaciones antes de desplegar el sistema en un entorno de producción, y realizar revisiones de seguridad periódicas para garantizar la protección continua del sistema.

Documentado por: Luis, Andrés y Sebastián

Fecha: [30 / 08 / 2024]

Link del proceso realizado :

<https://www.youtube.com/watch?v=479kIOIrwm0> [Hacking Etic Controlled Attack, nikto tool]